lab_08_Feltrin

Mariana Zamith Feltrin

```
library(reticulate)
```

```
py_require("polars")
py_require("matplotlib")
py_require("plotnine")
py_require("pandas")
py_require("pyarrow")
```

```
import polars as pl

#importando os bancos de dados

basics = pl.read_csv(
    "title.basics0.tsv.gz",
    separator="\t",
    null_values="\\N"
)

ratings = pl.read_csv(
    "title.ratings.tsv.gz",
    separator="\t",
    null_values="\\N"
)

principals = pl.read_csv(
    "title.principals0.tsv.gz",
    separator="\t",
    null_values="\\N"
)
```

Exercício 1

```
# Filtrando apenas os filmes
filmes = basics.filter(pl.col("titleType") == "movie")

# Unindo com a tabela de notas e ordenando
top5_filmes = (
    filmes.join(ratings, on="tconst", how="inner")
    .sort(["averageRating", "numVotes"], descending=[True, True])
    .select(["primaryTitle", "averageRating", "numVotes"])
    .head(5)
)

print(top5_filmes)
```

```
shape: (5, 3)

 primaryTitle              averageRating  numVotes
 ---                       ---            ---
 str                       f64            i64

 Kaveri                    10.0           1023
 Kurukku                   10.0           451
 Jedal Dar Omghe 30 Metri  10.0           142
 Sargashte                 10.0           134
 Gorgeous Rascal           10.0           115
```

Exercício 2

```
# Considerando apenas filmes com averageRating > 8, organizando e agrupando por gênero

genero_frequente = (
    filmes.join(ratings, on="tconst", how="inner")
    .filter(pl.col("averageRating") > 8)
    .with_columns(pl.col("genres").str.split(","))
    .explode("genres")
    .group_by("genres")
```

```
    .len()
    .sort("len", descending=True)
    .head(1)
)

print(genero_frequente)
```

```
shape: (1, 2)

 genres       len
 ---          ---
 str          u32

 Documentary  10564
```

Exercício 3

```
# Filtrando os filmes, unindo e agrupando os elencos

elenco = principals.filter(pl.col("category").is_in(["actor", "actress"]))

filmes_boa_nota = (
    filmes.join(ratings, on="tconst", how="inner")
    .filter(pl.col("averageRating") > 7.5)
)

top3_atores = (
    filmes_boa_nota.join(elenco, on="tconst", how="inner")
    .group_by("nconst")
    .len()
    .sort("len", descending=True)
    .head(3)
)

print(top3_atores)
```

```
shape: (3, 2)
```

```
 nconst      len
 ---         ---
 str         u32

 nm0004660   231
 nm0595934   155
 nm3183374   124
```

```
Sys.setenv(TZ = "America/Sao_Paulo")
Sys.time()
```

```
[1] "2026-10-13 08:56:51 -03"
```